Unitree Go2 の歩行とジャンプを強化学習で獲得する

— 報酬の「仮説 → 学習 → 評価 → 原因の分析」を 20 回以上くり返した記録 —
Physical AI 応用1講座 最終課題 説明資料　／　コード: https://github.com/ikechi1/physical_ai_go2

1. 課題の概要と目的

四脚ロボット Unitree Go2 に、**速度指令どおりに歩きながら、「跳べ」の合図でジャンプする**動きを、強化学習（PPO）で身につけさせる。最終的な目標は「障害物まで歩いて行き、跳び越える」ことである。

ジャンプは歩行と違い、「どういう動きが良いジャンプか」を報酬として書き下すのが難しい。報酬を少し変えるだけで、ロボットは設計者の意図しない「抜け穴」を見つけて報酬を稼ぐ。そこで本課題では、**報酬の仮説を自分で立て、学習結果を数値と映像で評価し、なぜその動きになったかを報酬の損得から説明する**、という手順を繰り返すことを目的とした。

2. 環境と手法

項目	内容
シミュレータ	Genesis（Apple Silicon の GPU で 4096 台を並列に動かして学習、制御周期 50Hz）
ロボット	Unitree Go2（12 関節、PD 制御で関節の目標角度を与える）
学習アルゴリズム	rsl-rl の PPO（方策・価値関数とも 512-256-128 の全結合ネットワーク）。1 回の学習は 500 イテレーション（約 4,900 万ステップ、約 15 分）
行動（12 次元）	各関節の目標角度（基準姿勢からのずれ）
観測（47 次元）	胴体の角速度 3、重力の向き 3、**指令 4（前後・左右の速度、旋回速度、跳べフラグ）**、関節角度 12、関節速度 12、前回の行動 12、**ジャンプの進み具合 1（0〜1）**
跳べフラグ	4 秒ごとの指令の引き直しで、確率 `jump_prob` で 1 になり、1.0 秒間保持。キーボード操作（`j` キー）でも立てられる
終了条件	胴体の傾き（roll / pitch）が 10° を超える、または 20 秒経過

最終的に使った報酬（その場ジャンプ、仮説20）

報酬	重み	内容
`tracking_lin_vel` / `tracking_ang_vel`	1.0 / 0.2	速度指令との近さ（歩行の報酬）
`lin_vel_z` / `base_height`	-1 / -500	上下の速度、体高と 0.3m の差の罰。**フラグが立っている間はかけない**
`action_rate` / `similar_to_default`	-0.005 / -0.1	動きの激しさ、基準姿勢からのずれの罰
`jump_up_vel`	10	上向きの速度（フラグから 0.5 秒以内、体高 0.45m 未満、2.0m/s で頭打ち）
`jump_air_time`	20	4 本とも浮いている時間のうち 0.2 秒を超えた分（フラグ後の 1 回目の飛行だけ）
`jump_one_flight`	200	フラグの終わりに、跳び上がった回数がちょうど 1 回なら 1 回だけ支払う（+4）
`landed_stance`	8	1 回目の飛行のあと、4 本とも接地している時間
`base_contact`	-200	胴体が床に触れたら罰

（報酬は毎ステップ「重み × 0.02 秒」で足される。歩行＋ジャンプではこれに `termination`（転倒で 1 回 -10）を加えた。）

3. 工夫した点

- **「跳べフラグ」と「ジャンプの進み具合」を観測に入れた。**1 つのポリシーで、歩行とジャンプを指令によって切り替えられる。進み具合（0〜1）があることで、「今は踏み切りの時期か、着地の時期か」をポリシーが区別できる。
- **報酬は「仮説」として 1 つずつ変え、毎回同じ方法で評価した。**評価では行動のばらつきを除いたポリシーを 32 台 × 12 秒動かし、跳び上がった回数、最高到達点、転倒回数、立っているときの体高を測った。数値で分からないときは、1 台の動きを 0.02 秒ごとに書き出して（足の接地・体高・上下速度）原因を調べた。
- **結果を「報酬の損得の比較」で説明した。**意図しない動きが出たとき、「その動きと、意図した動きで、それぞれ報酬がいくらになるか」を計算し、なぜロボットがそちらを選んだかを確かめてから次の仮説を立てた。
- **学習済みの歩行ポリシーの重みを、入力の数が違うネットワークに引き継いだ。**歩行のみのポリシー（観測 45 次元）の 1 層目に、跳べフラグとジャンプの進み具合の 2 列を 0 で足して 47 次元にした。読み込んだ直後は元の歩行と同じ動きをするので、歩行を一から覚え直す必要がない（W3 以降）。
- **終了の判定を報酬の計算より前に移した**ことで、「転倒した瞬間」に罰を与えられるようにした。
- キーボードで歩かせて跳ばせる操作プログラムと、字幕付きで動画を自動生成するプログラム（`make_video.py`）を用意し、どの学習結果もすぐ目で確認できるようにした。

4. 結果

4.1 歩行

歩行の報酬（速度指令との近さ＋姿勢・動きの罰）で 500 回学習し、前後・左右・旋回の速度指令に追従して歩けるようになった（`tracking_lin_vel` 約 0.93、転倒ほぼなし）。

4.2 その場ジャンプ（仮説 1〜20）

仮説	変更内容	結果
1	フラグ中は体高の目標を +15cm	背伸びするだけで跳ばない（最高 0.393m）
2	4 本とも浮いている時間に報酬	跳ばない。一度も浮かないので報酬が 0 のまま（まばらな報酬）
4	上向きの速度に報酬、フラグ中は体高・上下速度の罰を外す	初めて跳ぶ。最高 0.767m だが、32 台 × 12 秒で 64 回転倒
5	上向き速度の報酬を 0.3 秒以内・0.45m 未満に制限	転倒 0。1 回のフラグで 3.67 回跳ねる
6	滞空の報酬は 1 回目の飛行だけ	1 回跳び（最高 0.538m）。ただし普段から低くしゃがんで構える
7	フラグが 0 のときの体高の罰を -50 → -500	普段の姿勢が 0.237m → 0.280m に戻る。最高 0.415m
8	踏み切りの窓を 0.3 → 0.5 秒	胴体から床に落とし、その反動で上向き速度を稼ぐ。2 回跳び
9〜11	しゃがみ期を分ける、胴体の接地に罰、しゃがみ期の下向き速度に罰	胴体の接地は消えたが、時間をずらして同じ「落下＋反動」を続ける
12〜13	ちょうど 1 回跳んだら +4（`jump_one_flight`）	1 回跳びの割合が一時 25% まで上がるが定着しない
14	滞空の報酬をなくす／0.25 秒以上だけ払う	それでも落下する → 動機は滞空ではなく**上向き速度の反動**だと特定
15	上向き速度の重み 20 → 10	初めて、落下なしのきれいな 1 回跳び（最高 0.403m）
16〜18	滞空報酬の重み・下限・1 回目だけの調整	高く跳ぶ（最高 0.514m）が、着地の小さな跳ねで 2 回に
19	2 回目以降の飛行に罰	罰を恐れて跳ばなくなる（背伸び 0.345m）
20	19 の罰を外し、着地して 4 本足で立っている時間に報酬	300 回目以降、毎回ちょうど 1 回跳ぶ。最高 平均 0.494m／最大 0.518m、転倒 0、胴体の接地 0

（仮説3 は途中で止めたため結果なし。仮説 8〜20 は報酬の値を少しずつ変えたもので、詳細はリポジトリのコードのコメントに残した。）

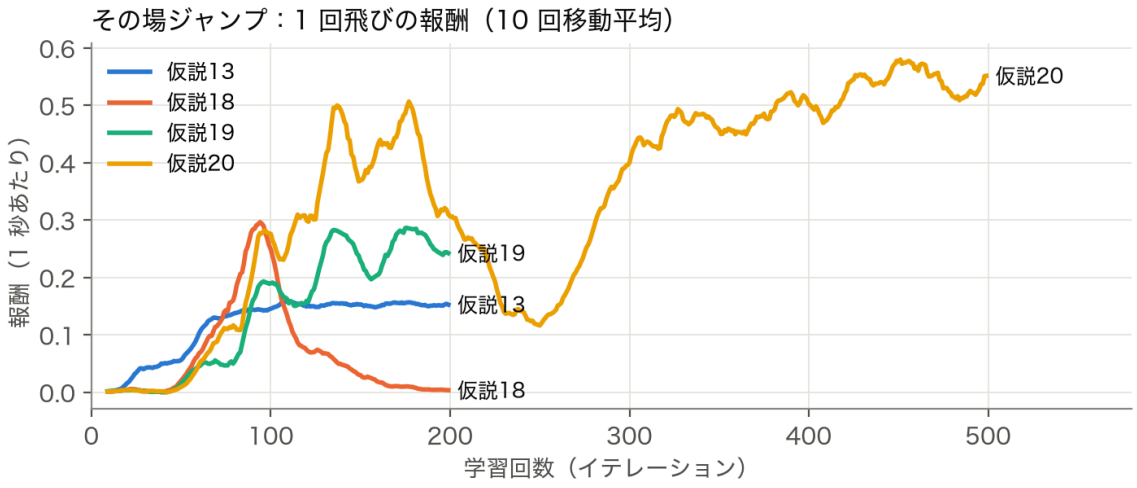


図1 「ちょうど 1 回跳んだら +4」の報酬の推移。仮説18 は途中で 2 回跳びに移って消え、仮説20 は 300 回目以降に定着した。

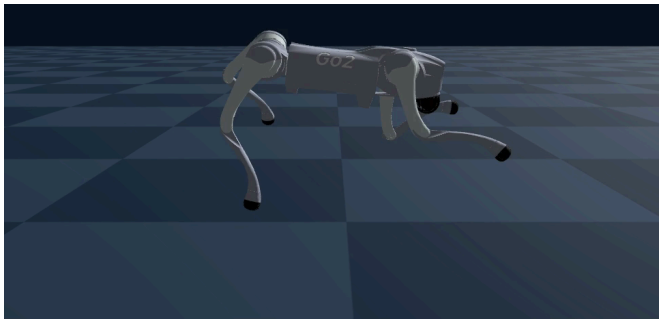


図2　仮説20 の踏み切り

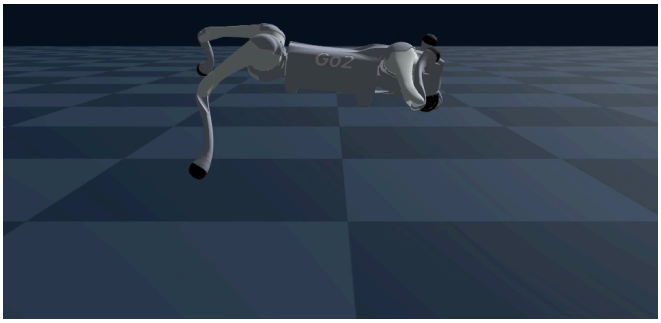


図3　仮説20 の最高点付近（4 本とも浮いている）

4.3 歩行＋ジャンプ（W1〜W4）

速度指令（前後 -0.5〜1.0m/s、左右 ±0.5m/s、旋回 ±1.0rad/s）を出しながら、跳べフラグも立てて学習した。

試行	変更内容	結果
W1	仮説20 の報酬をそのまま使い、`jump_prob` 0.3	「自殺」：出現直後の体高の罰（-500）から逃げるため、3 ステップで傾いてエピソードを終わらせる
W2	体高の罰を -50 に戻し、転倒に罰（1 回 -10）	自殺は消えたが、転ぶ危険のある歩く練習をせず、立ち止まる
W3	学習済みの歩行ポリシーの重みから開始	歩ける（指令 0.5 / 1.0m/s に対し 0.66 / 1.10m/s、転倒 0）。フラグが立っても跳ばない
W4	`jump_prob` 0.3 → 1.0（ジャンプの練習量を約 3 倍に）	歩きながら跳ぶ（最高 0.395m、転倒 0）。1 回のフラグで小さく約 3 回跳ねる。後退とその場での旋回ができなくなった

操作して確かめたところ、W4 では**後退とその場での旋回ができなくなっていた。**ジャンプなしで指令を固定し、16 台 × 6 秒で測った結果を下に示す（値は実際の前後の速度と旋回速度の平均）。

指令	歩行のみ	W3	W4
後退 vx = -0.5m/s	-0.49m/s	-0.38m/s	-0.00m/s
その場で左旋回 yaw = +1.0rad/s	+0.98rad/s	+0.69rad/s	+0.01rad/s
その場で右旋回 yaw = -1.0rad/s	-1.02rad/s	-0.10rad/s	-0.01rad/s
前進しながら左旋回 vx = 0.5、yaw = +1.0	0.51m/s、+0.97rad/s	0.66m/s、+0.94rad/s	0.61m/s、+0.72rad/s

歩行のみのポリシーはどの指令にも追従できていたが、ジャンプを加えて学習し直すと、W3 で右旋回が、W4 では後退とその場での旋回がほぼできなくなった（その場に立ったまま）。前進しながらの旋回は W4 でも残っている。転倒はどの場合も 0 回だった。

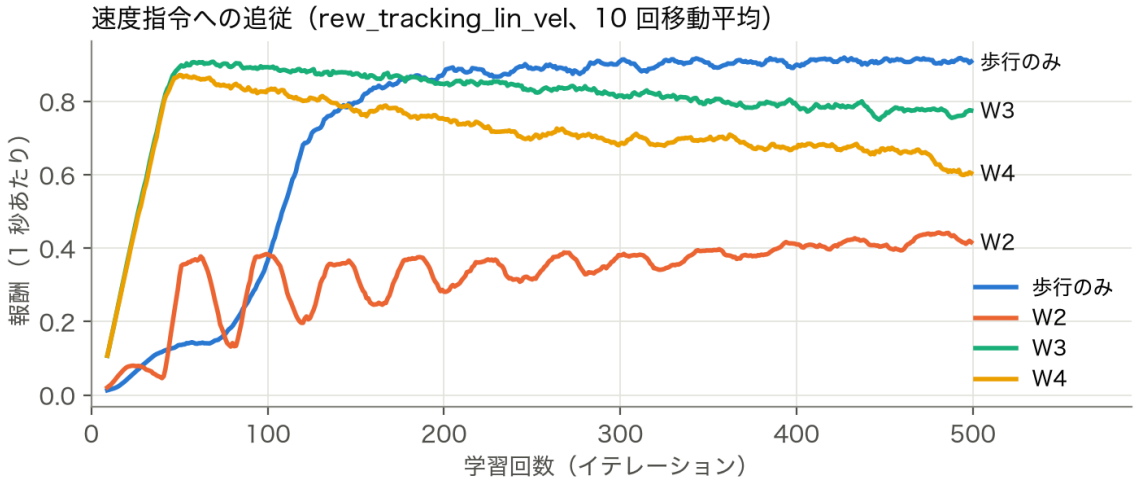


図4　速度指令への追従。W2 は「立ち止まっていても得られる値」（約 0.4）から上がらない。W3・W4 は歩行の重みから始めたので、最初から歩ける。

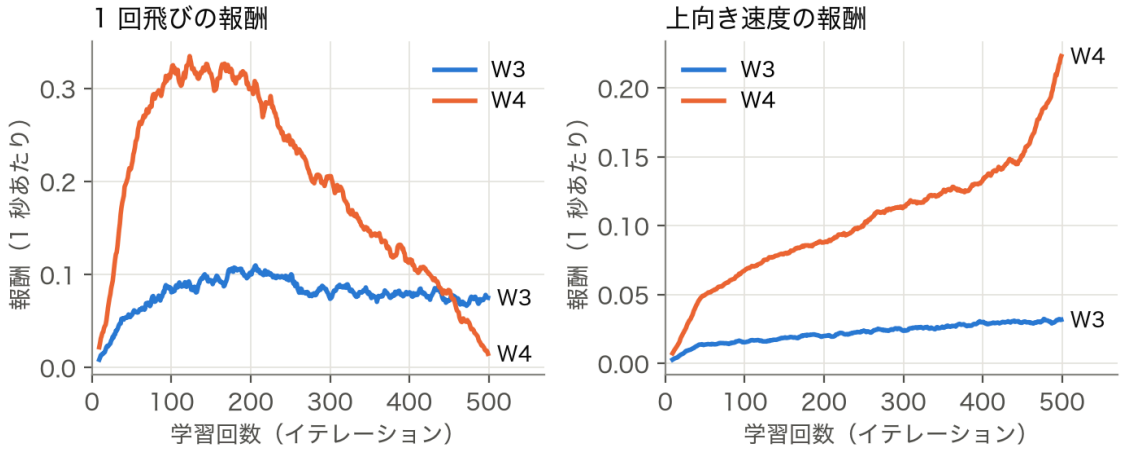


図5　W3 と W4 のジャンプの報酬。W4 では「1 回だけ跳ぶ」報酬が途中で減り、代わりに「上向き速度」の報酬が増えた（何度も跳ねて稼ぐ方へ移った）。

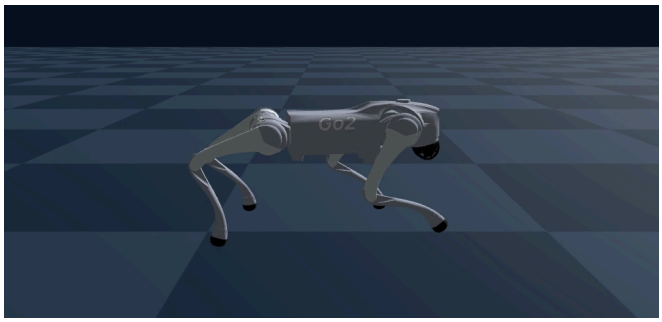


図6　W4：0.5m/s で歩行中

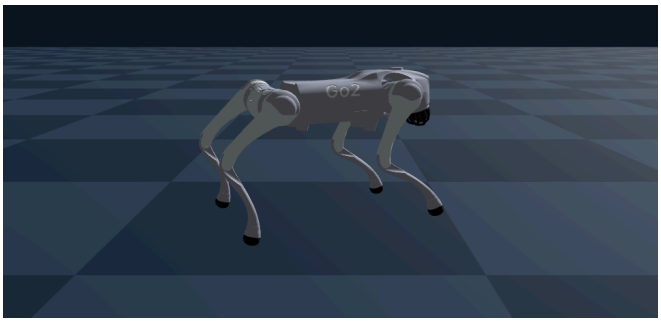


図7　W4：歩きながら跳んだ最高点付近（4 本とも浮いている、約 0.38m）

5. 考察

> **報酬は、設計者の意図ではなく「書いた式」どおりに最大化される。**今回見つかった抜け穴は、次の 4 つの型に分けられた。
>
> - **足し合わせると別の量になる**：上向き速度を毎ステップ足すと「上がった距離」になる。落下で一度下がってから跳ねても稼げる（仮説8）。
> - **数えられる報酬は、くり返して稼がれる**：飛行ごとに払う報酬は、何度も跳ねて稼がれる（仮説5、17、W4）。
> - **時間の窓は、タイミングをずらしてかわされる**：「しゃがみ期の下向き速度に罰」を入れると、小さく跳んでから罰の窓の外で落ちる（仮説11）。
> - **しきい値のある報酬は、まばらな報酬と同じ**：「0.2 秒以上浮いたら報酬」は、一度も達成しないうちは傾きが 0 で学習が進まない（仮説2、16）。

- **罰は「やってほしくないこと」だけでなく「挑戦すること」も止めてしまう。**2 回目の飛行への罰（仮説19）では跳ぶこと自体をやめ、転倒への罰（W2）では歩く練習をやめた。罰よりも、望ましい最終状態（着地して立つ）に報酬を与える方がうまくいった（仮説20）。
- **行動を選ぶ理由は、選択肢ごとの報酬の差で説明できる。**W1 の自殺は「生き続けて体高の罰を受け続ける」より「すぐ終わる」方が合計の罰が小さい、という計算で説明できた。
- **重みの大きさは影響の大きさではない。**ログの値は「1 エピソードの合計 ÷ 20 秒」で、条件付きの報酬は重みが大きくても実際の寄与は小さい。罰の効果は、ログでは「その行動が起きなくなった」としか見えないので、評価で動きを直接確かめる必要があった。
- 歩行とジャンプの同時学習では、**練習量の配分**（ジャンプの頻度）と**初期値**（歩行の重みから始める）が効いた。一方で、ジャンプの練習を増やすと歩行の追従が下がる（図4、W4）という取り合いも起きた。特に後退とその場での旋回は、歩行のみのポリシーではできていたのに、ジャンプを加えた学習で失われた（学習済みの動きを忘れる「破滅的忘却」に近い）。立ち止まっていても、速度の報酬は指令が 0 に近いほど一部もらえるため、まれな指令の練習が後回しになった可能性があるが、原因は確かめていない。

6. 今後の課題

- 歩行＋ジャンプでも「1 回だけ跳ぶ」ようにする。W4 の跳ねはどれも上向き速度の窓（0.5 秒）の中にあるので、窓の見直しや、その場ジャンプのポリシーから始める方法を試す。
- 歩行＋ジャンプで失われた後退とその場での旋回を取り戻す。ジャンプの練習を増やしても歩行の練習が減らないよう、フラグ中とそれ以外で報酬や練習量の配分を見直す。
- 障害物（箱）を置き、「歩いて近づき、跳び越える」タスクにする。障害物までの距離を観測に入れ、ポリシー自身が跳ぶタイミングを決めるようにする。
- 実機（Go2）への移植に向け、質量・摩擦のランダム化などで、シミュレーションと現実の差に強くする。

7. 提出物と再現方法

- コード：https://github.com/ikechi1/physical_ai_go2（`src/physical_ai_go2/`）
- 学習：`uv run go2-train-walk`、`uv run go2-train-jump`、`uv run go2-train-walk-jump -e go2-walk-jump-w4 --init-from logs/go2-walking/model_499.pt`
- 操作：`uv run go2-teleop -e go2-walk-jump-w4`（矢印キーで歩行、`j` で跳ぶ）
- 動画の生成：`uv run go2-make-video`（動画は等倍速、スローの場面は画面内に「1/4 倍速」と表示）
- AI の活用：ロボットの制御は強化学習（PPO）で学習したニューラルネットワークによる。また、実装と評価スクリプトの作成には AI コーディングアシスタントを使い、報酬の仮説は自分で立てた。